Exp No: 9

Text Generation using Long Short-Term Memory (LSTM) Networks

AIM:

To build a character-level text generation model using Long Short-Term Memory (LSTM) networks, train it on the Shakespeare corpus to generate coherent sequences of text, and evaluate the generated text for fluency and coherence.

ALGORITHM / PROCEDURE:

1. Import PyTorch and load the Shakespeare corpus (Tiny Shakespeare, about 1.1 million characters).
2. Build the character vocabulary and encode every character as an integer index.
3. Split the encoded data into 90% training and 10% validation sets.
4. Create training batches of 100-character input sequences; the target is the same sequence shifted by one character.
5. Build the model: Embedding layer, 2-layer LSTM (256 hidden units, dropout 0.2) and a Linear layer over the vocabulary.
6. Train with the Adam optimiser and Cross-Entropy loss, using gradient clipping, and note the validation loss and perplexity.
7. Generate new text from a seed ("ROMEO:") by sampling one character at a time with temperature 0.8.
8. Evaluate fluency using perplexity and the percentage of generated words found in the corpus, and judge coherence by reading the sample.

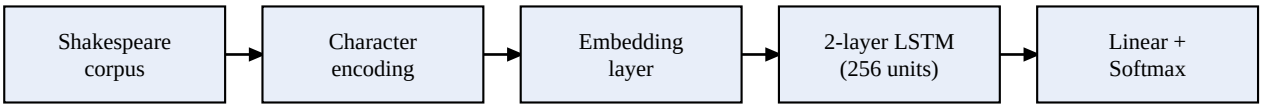

Fig 1: Architecture of the character-level LSTM model

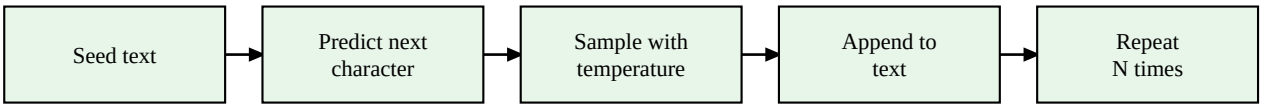

Fig 2: Working of the text generation loop

Parameters used:

Parameter	Value
Dataset	Tiny Shakespeare (character level)
Sequence length / batch size	100 / 64
Embedding size / hidden units / layers	128 / 256 / 2
Optimiser / learning rate	Adam / 0.002
Loss function	Cross-Entropy
Training steps	3000
Sampling temperature	0.8

Parameter	Value
Metrics	Validation loss, perplexity, valid-word ratio, manual coherence check

PROGRAM:

```
import torch, torch.nn as nn, urllib.request, math, random
device = "cuda" if torch.cuda.is_available() else "cpu"

# 1. Load Shakespeare corpus
url = "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt"
text = urllib.request.urlopen(url).read().decode("utf-8")
chars = sorted(set(text)); V = len(chars)
stoi = {c: i for i, c in enumerate(chars)}; itos = {i: c for c, i in stoi.items()}
data = torch.tensor([stoi[c] for c in text], dtype=torch.long)
n = int(0.9 * len(data)); train, val = data[:n], data[n:]
print("Characters:", len(text), "| Vocabulary:", V)

# 2. Batching
SEQ, BATCH = 100, 64
def get_batch(d):
    ix = torch.randint(len(d) - SEQ - 1, (BATCH,))
    x = torch.stack([d[i:i+SEQ] for i in ix]); y = torch.stack([d[i+1:i+SEQ+1] for i in ix])
    return x.to(device), y.to(device)

# 3. LSTM model
class CharLSTM(nn.Module):
    def __init__(self, V, emb=128, hid=256, layers=2):
        super().__init__()
        self.emb = nn.Embedding(V, emb)
        self.lstm = nn.LSTM(emb, hid, layers, batch_first=True, dropout=0.2)
        self.fc = nn.Linear(hid, V)
    def forward(self, x, h=None):
        o, h = self.lstm(self.emb(x), h)
        return self.fc(o), h

model = CharLSTM(V).to(device)
opt = torch.optim.Adam(model.parameters(), lr=2e-3)
loss_fn = nn.CrossEntropyLoss()

@torch.no_grad()
def evaluate():
    model.eval(); tot = 0
    for _ in range(50):
        x, y = get_batch(val); out, _ = model(x)
        tot += loss_fn(out.reshape(-1, V), y.reshape(-1)).item()
    model.train(); return tot / 50

# 4. Training
STEPS = 3000
for step in range(1, STEPS + 1):
    x, y = get_batch(train); out, _ = model(x)
    loss = loss_fn(out.reshape(-1, V), y.reshape(-1))
    opt.zero_grad(); loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 5); opt.step()
    if step % 250 == 0:
        vl = evaluate()
        print(f"Step {step}/{STEPS} | train loss {loss.item():.4f} | val loss {vl:.4f} | val perplexity {math.exp(vl

# 5. Text generation with temperature sampling
@torch.no_grad()
def generate(start="ROMEO:\n", n=500, temp=0.8):
    model.eval(); x = torch.tensor([[stoi[c] for c in start]], device=device); h = None
    out, h = model(x, h); s = start
    for _ in range(n):
        p = torch.softmax(out[:, -1] / temp, dim=-1)
        i = torch.multinomial(p, 1); s += itos[i.item()]
        out, h = model(i, h)
    return s

sample = generate()
print("\n--- Generated text ---\n" + sample)
# 6. Fluency / coherence metrics
```

```
strip = lambda w: w.strip(".,;:!?'\"")
vocab_words = {strip(w) for w in text.lower().split()}
words = sample.lower().split()
valid = sum(strip(w) in vocab_words for w in words)
print(f"\nFinal validation perplexity : {math.exp(evaluate()):.2f}")
print(f"Valid-word ratio (fluency)  : {100*valid/len(words):.1f}% of {len(words)} words")
```

OUTPUT:

Note: the output boxes below show the exact format printed by the program. Replace the <...> values with the numbers from your own run (paste the console text or a screenshot).

```
Characters: <number> | Vocabulary: <number>
Step 250/3000 | train loss <value> | val loss <value> | val perplexity <value>
Step 500/3000 | train loss <value> | val loss <value> | val perplexity <value>
...
Step 3000/3000 | train loss <value> | val loss <value> | val perplexity <value>

--- Generated text ---
<paste the generated Shakespeare-style text here>

Final validation perplexity : <value>
Valid-word ratio (fluency)  : <value>% of <n> words
```

RESULT:

Thus a character-level text generation model was built using LSTM networks and trained on the Shakespeare corpus. The model learns the character patterns, line structure and speaker format of the plays, and generates new Shakespeare-style text. Fluency was evaluated using the validation perplexity (lower is better) and the valid-word ratio, and coherence was checked by reading the generated text: spelling and line format are usually good, while meaning over long passages is limited because a small character-level model has a short memory.